29.1 技能平台设计

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~400k tokens，输出 ~7k tokens（本节）

LLM 本身是通用的——它什么都能聊，但什么都不精通。要让 Agent 在特定领域（天气查询、GitHub 操作、智能家居控制）变得专业，需要给它注入**领域知识和操作指南**。OpenClaw 的技能系统（Skills）正是为此而设计：一套可热插拔、自描述、自动发现的 Agent 能力扩展机制。

29.1.1 什么是技能：可热插拔的 Agent 能力模块

设计理念

技能的核心思想很简单：**一个 Markdown 文件就是一项技能**。技能文件（通常名为 `SKILL.md` ）包含两部分：

1. **Front Matter**（YAML 头部）——结构化元数据：名称、描述、依赖要求、安装方式
2. **正文**——Agent 要学习的领域知识：API 用法、命令示例、最佳实践

```
---
name: weather
description: Get current weather and forecasts (no API key required).
metadata: { "openclaw": { "emoji": "🌤", "requires": { "bins": ["curl"] } } }
---

# Weather

Two free services, no API keys needed.

## wttr.in (primary)

Quick one-liner:
```bash
curl -s "wttr.in/London?format=3"
```
```

Agent 在接收到用户请求时，系统会将符合条件的技能内容注入到系统提示词（System Prompt）中。Agent 不需要"调用"技能——它只需要**阅读**技能文件中的指引，然后使用已有的工具（如 `bash` ）执行操作。

> **衍生解释**：这种设计与传统的"插件 API"截然不同。传统插件定义 API 端点供程序调用；而技能是面向 LLM 的"文档"，通过自然语言指导 Agent 行为。这利用了 LLM 的核心优势——理解和遵循自然语言指令。

技能 vs 工具

OpenClaw 中的"技能"和"工具"是不同概念：

维度	技能（Skill）	工具（Tool）
本质	Markdown 文档	可执行的函数
消费者	LLM（通过系统提示词阅读）	LLM（通过 function calling 调用）
扩展方式	写一个 `.md` 文件	编写 TypeScript 代码并注册
示例	weather、github、discord	bash、file_read、memory_search

技能告诉 Agent **怎么做**，工具给 Agent **能力去做**。一个天气技能告诉 Agent "用 `curl` 调用 wttr.in"， `bash` 工具让 Agent 能执行 `curl` 命令。

29.1.2 技能类型：Bundled / Managed / Workspace / Extra

OpenClaw 支持四种来源的技能，按优先级从低到高排列：

类型	目录位置	来源标识	说明
Extra	配置的额外目录	`openclaw-extra`	插件或自定义目录提供的技能
Bundled	`<package>/skills/`	`openclaw-bundled`	随 OpenClaw 安装包分发的内置技能
Managed	`~/.openclaw/skills/`	`openclaw-managed`	通过 ClawHub 安装的托管技能
Workspace	`<workspace>/skills/`	`openclaw-workspace`	项目级别的自定义技能

优先级体现为：**同名技能，高优先级覆盖低优先级**。如果项目目录下有一个自定义的 `weather` 技能，它会覆盖内置的 `weather` 技能。

```
// src/agents/skills/workspace.ts（简化）

const merged = new Map<string, Skill>();
// 优先级：extra < bundled < managed < workspace
for (const skill of extraSkills)    merged.set(skill.name, skill);
for (const skill of bundledSkills)  merged.set(skill.name, skill);
for (const skill of managedSkills)  merged.set(skill.name, skill);
for (const skill of workspaceSkills) merged.set(skill.name, skill);
```

内置技能（Bundled）

OpenClaw 随包附带 52 个内置技能，覆盖开发工具、通信平台、智能家居、媒体处理等领域：

类别	代表性技能	说明
开发工具	github, coding-agent, tmux	Git 操作、子 Agent 编排、终端会话
通信平台	discord, slack, imsg, bluebubbles	聊天平台集成
笔记/知识	obsidian, notion, bear-notes, apple-notes	各类笔记应用
智能家居	openhue, sonoscli	Hue 灯光、Sonos 音响

类别	代表性技能	说明
AI/模型	gemini, oracle, openai-image-gen	模型调用、图像生成
媒体	spotify-player, video-frames, camsnap	音乐、视频、摄像头
工具	weather, summarize, healthcheck	天气、摘要、健康检查
生态	clawhub, skill-creator	技能注册中心、技能创作

内置技能目录解析

`resolveBundledSkillsDir` 负责定位内置技能的物理目录，支持多种安装场景：

```
// src/agents/skills/bundled-dir.ts（简化）

export function resolveBundledSkillsDir(): string | undefined {
  // 1. 环境变量覆盖
  const override = process.env.OPENCLAW_BUNDLED_SKILLS_DIR;
  if (override) return override;

  // 2. 编译后二进制文件：查找可执行文件旁边的 skills/ 目录
  const sibling = path.join(path.dirname(process.execPath), "skills");
  if (fs.existsSync(sibling)) return sibling;

  // 3. npm/开发模式：从包根目录查找 skills/
  const packageRoot = resolveOpenClawPackageRootSync();
  if (packageRoot) {
    const candidate = path.join(packageRoot, "skills");
    if (looksLikeSkillsDir(candidate)) return candidate;
  }

  // 4. 向上遍历最多 6 层目录查找
  // ...
  return undefined;
}
```

`looksLikeSkillsDir` 通过检查目录中是否存在 `.md` 文件或包含 `SKILL.md` 的子目录来判断。

29.1.3 技能加载与快照

加载流程

`loadSkillEntries` 是技能加载的核心函数。它从四个来源加载技能，解析 Front Matter，提取元数据：

```
// src/agents/skills/workspace.ts（简化）

function loadSkillEntries(workspaceDir, opts?): SkillEntry[] {
  // 1. 从各来源加载原始 Skill 对象
  const bundledSkills  = loadSkillsFromDir({ dir: bundledSkillsDir, source: "openclaw-b
  const managedSkills  = loadSkillsFromDir({ dir: managedSkillsDir, source: "openclaw-m
  const workspaceSkills = loadSkillsFromDir({ dir: workspaceSkillsDir, source: "opencla
  const extraSkills    = extraDirs.flatMap(dir => loadSkillsFromDir({ dir, source: "ope

  // 2. 按优先级合并（同名覆盖）
  const merged = new Map<string, Skill>();
  // ... extra < bundled < managed < workspace

  // 3. 为每个技能解析 Front Matter 和元数据
  return Array.from(merged.values()).map(skill => {
    const raw = fs.readFileSync(skill.filePath, "utf-8");
    const frontmatter = parseFrontmatter(raw);
    return {
      skill,
      frontmatter,
      metadata: resolveOpenClawMetadata(frontmatter),
      invocation: resolveSkillInvocationPolicy(frontmatter),
    };
  });
}
```

每个 `SkillEntry` 包含四个层次的信息：

```
type SkillEntry = {
  skill: Skill;                        // 基础信息：name, description, filePath, baseDir
  frontmatter: ParsedSkillFrontmatter; // 原始 Front Matter 键值对
  metadata?: OpenClawSkillMetadata;    // 解析后的 OpenClaw 特定元数据
  invocation?: SkillInvocationPolicy;  // 调用策略
};
```

资格过滤

并非所有加载的技能都会被激活。 `shouldIncludeSkill` 实现了多维度的资格检查：

```
// src/agents/skills/config.ts（简化）

export function shouldIncludeSkill(params: { entry, config, eligibility }): boolean {
  // 1. 配置禁用检查
  if (skillConfig?.enabled === false) return false;

  // 2. 内置技能白名单检查
  if (!isBundledSkillAllowed(entry, allowBundled)) return false;

  // 3. 操作系统兼容性检查
  if (osList.length > 0 && !osList.includes(process.platform)) return false;

  // 4. always 标记：无条件激活
  if (entry.metadata?.always === true) return true;

  // 5. 必需二进制文件检查（PATH 中是否存在）
  for (const bin of requiredBins) {
    if (!hasBinary(bin)) return false;
  }

  // 6. 任一二进制文件检查（至少有一个）
  if (requiredAnyBins.length > 0 && !requiredAnyBins.some(bin => hasBinary(bin))) retur

  // 7. 环境变量检查
  for (const envName of requiredEnv) {
    if (!process.env[envName] && !skillConfig?.env?.[envName]) return false;
  }

  // 8. 配置路径检查
  for (const configPath of requiredConfig) {
    if (!isConfigPathTruthy(config, configPath)) return false;
  }

  return true;
}
```

这就是为什么用户安装 OpenClaw 后，只会看到与其环境匹配的技能——没有安装 `gh` CLI 的用户不会看到 `github` 技能，没有配置 Discord Token 的用户不会看到 `discord` 技能。

技能快照

技能快照（ `SkillSnapshot` ）是某一时刻所有激活技能的静态表示，用于注入到 Agent 的系统提示词中：

```
// src/agents/skills/types.ts

export type SkillSnapshot = {
  prompt: string;                                    // 格式化后的技能提示词
  skills: Array<{ name: string; primaryEnv?: string }>; // 激活的技能列表
  resolvedSkills?: Skill[];                          // 完整的 Skill 对象
  version?: number;                                  // 快照版本（用于缓存）
};
```

`buildWorkspaceSkillSnapshot` 构建快照的过程：加载 → 过滤 → 排除 `disableModelInvocation` 的技能 → 调用 `formatSkillsForPrompt` 生成提示文本。

技能命令规格

每个技能还可以注册为斜杠命令（ `/weather` 、 `/github` ），让用户直接调用。 `buildWorkspaceSkillCommandSpecs` 为每个可调用的技能生成命令规格：

```
// src/agents/skills/workspace.ts（简化）

function buildWorkspaceSkillCommandSpecs(workspaceDir, opts?): SkillCommandSpec[] {
  const eligible = filterSkillEntries(entries, config);
  const userInvocable = eligible.filter(e => e.invocation?.userInvocable !== false);

  return userInvocable.map(entry => ({
    name: sanitizeSkillCommandName(entry.skill.name), // 小写+下划线，≤32字符
    skillName: entry.skill.name,
    description: entry.skill.description,
    dispatch: resolveCommandDispatch(entry),           // 可选的确定性调度
  }));
}
```

命令名经过 `sanitizeSkillCommandName` 处理（只保留 `[a-z0-9_]` ），并通过 `resolveUniqueSkillCommandName` 确保唯一性。技能的 Front Matter 还可以通过 `command-dispatch: tool` + `command-tool: <toolName>` 指定确定性调度——即命令不经过 LLM 推理，直接调用指定工具。

本节小结

1. **技能本质是 Markdown 文档**，通过 Front Matter 声明元数据，正文提供领域知识，注入系统提示词后指导 Agent 行为。
2. **四种来源**按优先级排列：Extra < Bundled < Managed < Workspace，同名技能高优先级覆盖低优先级。
3. **资格过滤**检查八个维度：配置禁用、白名单、OS 兼容性、 `always` 标记、必需二进制、环境变量、配置路径，确保只激活与当前环境匹配的技能。
4. **技能快照**是激活技能的静态表示， `formatSkillsForPrompt` 将其格式化为系统提示词的一部分。
5. **52 个内置技能**覆盖开发、通信、智能家居、媒体等领域，每个技能文件独立自包含，便于理解和扩展。